Smart Inventory Management System

Production Microservices Architecture, Kubernetes Orchestration & Jenkins CI/CD Pipeline

---

Author: Mohamed (Moha-sys) | **Program:** NTI DevOps & Cloud Native
GitHub: https://github.com/Moha-sys/smart-inventory | **Docker Hub:** mohadevv
Date: November 2026 | **Pipeline Status:** SUCCESS (Build #3)

1. Executive Summary & Architecture Overview

The Smart Inventory Management System is an enterprise-grade, distributed microservices platform engineered to deliver high-availability inventory tracking, automated low-stock alerting, and seamless zero-downtime continuous delivery. The system decouples core business logic into specialized containerized workloads orchestrated on Kubernetes, automated through Ansible, and delivered via an automated GitHub-triggered Jenkins CI/CD pipeline.

Component	Implementation Technology	High Availability & Replicas	Network Exposure
Frontend (Client)	React 18 + Vite (Served by Nginx)	3 Pod Replicas	ClusterIP (80) & NodePort (30080)
Backend (Server)	Node.js + Express (REST API)	3 Pod Replicas	ClusterIP (3000)
Worker (Alerts)	Node.js Poller + Webhook Daemon	1 Pod Replica	ClusterIP (3001)
Database	PostgreSQL 16 Alpine	1 Pod + 2Gi PVC (Bound)	ClusterIP (5432)
Ingress Controller	NGINX Ingress (Regex Rewrite)	Unified Routing Gateway	Port 80 / 443

2. Microservices Architecture & Local Docker Stack

Each service runs in an isolated, multi-stage Docker container built for minimum attack surface and rapid start times. In local testing, docker-compose.yml connects all containers via an internal network with Postgres healthchecks:

Docker Compose Verification (All Containers Healthy):

```
NAME                                IMAGE                                       STATUS                    PORTS
smart-inventory-frontend-1          mohadevv/smart-inventory-client:latest      Up 45 minutes (healthy)   0.0.0.0:8081->80/t
smart-inventory-inventory-service-1 mohadevv/smart-inventory-server:latest      Up 45 minutes (healthy)   0.0.0.0:3000->3000
smart-inventory-alert-service-1     mohadevv/smart-inventory-worker:latest      Up 45 minutes (healthy)   0.0.0.0:3001->3001
smart-inventory-postgres-1          postgres:16-alpine                          Up 45 minutes (healthy)   5432/tcp
```

Reverse-Proxy Routing Architecture:

- **Client SPA Routing:** Browser requests hit the React SPA on port 8081. Nginx serves pre-compiled static assets with 1-year immutable caching.
- **API Reverse-Proxy:** Nginx proxies /api/inventory/ to http://inventory-service:3000/ and /api/alerts/ to http://alert-service:3001/, stripping the prefix so Express routes receive pure endpoints (/items, /alerts). This eliminates all Cross-Origin Resource Sharing (CORS) complications.
- **Background Daemon:** alert-service polls /low-stock every 15,000ms. If item quantity ≤ reorder_level, it automatically inserts an active alert into PostgreSQL.

3. Kubernetes Production Topology & Manifest Specifications

The cluster topology strictly follows high-availability production best practices. All manifests are stored in the k8s/ directory and follow standardized naming conventions:

Manifest File	Kubernetes Kind	Key Configuration	Resilience & Security
client-deployment.yaml	Deployment	replicas: 3, Port 80, node-toleration	Probes on /healthz; limits 250m/256Mi
client-cluster-ip-service.yaml	Service	type: ClusterIP, Port 80 -> 80	Internal load balancer for client pods
client-nodeport-service.yaml	Service	type: NodePort, Port 80 -> 30080	External direct access for testing
server-deployment.yaml	Deployment	replicas: 3, Port 3000, env from secrets	HTTP readiness/liveness on /health
server-cluster-ip-service.yaml	Service	type: ClusterIP, Port 3000 -> 3000	Internal DNS: server-cluster-ip-service
worker-deployment.yaml	Deployment	replicas: 1, Port 3001, INVENTORY_URL	HTTP probes on /health (Port 3001)
worker-cluster-ip-service.yaml	Service	type: ClusterIP, Port 3001 -> 3001	Internal DNS: worker-cluster-ip-service
postgres-deployment.yaml	Deployment	replicas: 1, subPath: postgres	pg_isready exec liveness & readiness
postgres-cluster-ip-service.yaml	Service	type: ClusterIP, Port 5432 -> 5432	Internal database endpoint
database-persistent-volume.yaml	PersistentVolume	capacity: 2Gi, Retain, HostPath	Path: /home/moha/data/... (777)
database-persistent-volume-claim.yaml	PVC	capacity: 2Gi, ReadWriteOnce	Bound to database-persistent-volume
secret.yaml	Secret	POSTGRES_USER, POSTGRES_PASSWORD	Encrypted credentials injection
configmap.yaml	ConfigMap	DB_HOST, DB_PORT, DB_NAME, POLL_INTERVAL	Decoupled 12-factor configuration
ingress-service.yaml	Ingress	ingressClassName: nginx, rewrite-target: /$1	Unified reverse-proxy entry point

Live Kubernetes Cluster Status Output:

```
NAME                                      READY   STATUS    RESTARTS   AGE    IP               NODE
pod/client-deployment-5f589bf95c-25knn    1/1     Running   0          5m     192.168.219.88   master
pod/client-deployment-5f589bf95c-c2btt    1/1     Running   0          5m     192.168.219.86   master
pod/client-deployment-5f589bf95c-m9h5c    1/1     Running   0          5m     192.168.219.89   master
pod/postgres-deployment-7d4b7ff494-qqqbc  1/1     Running   0          50m    192.168.219.126  master
pod/server-deployment-676f4d5bcc-fb6kq    1/1     Running   0          5m     192.168.219.85   master
pod/server-deployment-676f4d5bcc-glsq9    1/1     Running   0          5m     192.168.219.91   master
pod/server-deployment-676f4d5bcc-ngjj4    1/1     Running   0          5m     192.168.219.90   master
pod/worker-deployment-7d89944c8d-gf4mg    1/1     Running   0          5m     192.168.219.87   master

NAME                                                  STATUS   VOLUME                      CAPACITY   ACCESS MODES
persistentvolumeclaim/database-persistent-volume-claim   Bound    database-persistent-volume   2Gi        RWO
```

4. Ansible Automation Engine

Ansible eliminates fragile manual scripts by enforcing declarative, idempotent infrastructure orchestration. The suite consists of two playbooks:

- **ansible/playbooks/setup-environment.yml:** Detects operating system (Debian/Ubuntu vs. RedHat/Fedora), installs Docker Engine, configures daemon service, downloads architecture-specific kubectl (ARM64/AMD64), installs Python Kubernetes modules, and deploys NGINX Ingress Controller.
- **ansible/playbooks/deploy-k8s.yml:** Invoked directly by Jenkins during CI/CD. Sequentially applies Secrets, ConfigMaps, Persistent Volume Claims, ClusterIP Services, Deployments, and Ingress. It injects the build tag dynamically (image_tag=${BUILD_NUMBER}), triggers kubectl rollout restart, and monitors rollout completion within a 120s timeout.

Ansible Execution Recap (Clean Green Run):

```
PLAY RECAP *********************************************************************************************
localhost                  : ok=26   changed=6    unreachable=0    failed=0    skipped=0    rescued=0    ignored=0
```

5. Full Jenkins CI/CD Pipeline Automation

The automated delivery pipeline connects developer commits to zero-downtime Kubernetes rollouts using **Jenkins**, **GitHub SCM**, **Docker Hub**, and **Ansible**.

Jenkinsfile Stage-by-Stage Breakdown:

1. **Checkout SCM:** Pulls the latest commit from https://github.com/Moha-sys/smart-inventory.git on branch main using credentials jenkins.
2. **Build Docker Images:** Concurrently builds mohadevv/smart-inventory-client, server, and worker, tagging each with both ${BUILD_NUMBER} and latest.
3. **Push to Docker Registry:** Authenticates with Docker Hub using Jenkins credentials ID docker and securely pushes all tagged images.
4. **Deploy to Kubernetes via Ansible:** Executes ansible-playbook -i ansible/inventory.ini ansible/playbooks/deploy-k8s.yml --extra-vars "image_tag=${BUILD_NUMBER} dockerhub_username=mohadevv".
5. **Verify Rollout:** Runs kubectl rollout status across all deployments to verify pod health before declaring success.
6. **Post Actions:** Automatically runs docker logout to clean up runner credentials.

Jenkins Build #3 Verification (Status: SUCCESS):

```
Build URL      : http://localhost:8080/job/smart-inventory/3/
Result         : SUCCESS (100% Green)
Duration       : 143.318 seconds
SCM Revision   : Git commit 2fdd803 from https://github.com/Moha-sys/smart-inventory.git
Published Tags : mohadevv/smart-inventory-client:3, server:3, worker:3 (and :latest)
Cluster Update : 8 Pods upgraded with zero downtime
```

6. Functional Testing & Data Persistence Proof

To verify end-to-end functionality, test items were created via the API, written to PostgreSQL, and processed by the background alert worker:

1. Creating Low-Stock Item via API:

```
curl -s -X POST http://localhost:3000/items -H 'Content-Type: application/json' \
  -d '{"sku":"SRV-DELL-R750","name":"PowerEdge R750","category":"Hardware","quantity":4,"reorder_level":10,"unit_price":450

Response: {"id":2,"sku":"SRV-DELL-R750","name":"PowerEdge R750","quantity":4,"reorder_level":10,"unit_price":"4500.00"}
```

2. Automatic Alert Raised by Background Worker:

```
curl -s http://localhost:3001/alerts

Response:
[{"id":2,"item_id":2,"sku":"SRV-DELL-R750","quantity":4,"reorder_level":10,"severity":"warning","status":"active"}]
```

3. Kubernetes NodePort Frontend Response:

```
curl -i http://localhost:30080/healthz
HTTP/1.1 200 OK
Server: nginx/1.27.5
Content-Type: application/octet-stream
ok
```

7. Production Operations & Cheatsheet

Cluster Inspection:
kubectl get all,pvc,ingress -l app=smart-inventory -o wide
Manual Rollout Restart:
kubectl rollout restart deployment/client-deployment deployment/server-deployment
Rollout Rollback (Instant Undo):
kubectl rollout undo deployment/client-deployment
Live Pod Log Streaming:
kubectl logs -l component=server -f --tail=50

Report Generated Successfully — Complete verification of Docker, Kubernetes, Ansible & Jenkins.